

GPUMODE Lecture Study Notes: Lecture 23

Tensor Cores

Core Problem the Lecture Addresses

This lecture explains how to program NVIDIA Tensor Cores effectively through CUTLASS and CuTe, with a particular focus on Hopper GPUs. The central issue is that Tensor Cores provide extremely high matrix multiplication throughput, but reaching that throughput requires careful control over memory movement, tiling, layouts, synchronization, and software pipelining.

At the mathematical level, matrix multiplication is simple. Given

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the output is

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

However, a GPU implementation must map this computation onto a hierarchy of thread blocks, warps, warp groups, shared memory, registers, global memory, and Tensor Core instructions. The hard part is not the formula; the hard part is the mapping.

Tensor Cores are extremely fast matrix engines. The performance problem is feeding them with data fast enough while avoiding synchronization stalls, layout mistakes, bank conflicts, and excessive address arithmetic.

Required Background

GEMM

The standard GEMM operation is

$$D = \alpha AB + \beta C.$$

Elementwise, this is

$$D_{m,n} = \alpha \sum_{k=0}^{K-1} A_{m,k} B_{k,n} + \beta C_{m,n}.$$

The multiplication and accumulation phase is called the mainloop. The post-processing phase, such as scaling by α , adding βC , adding bias, applying activation functions, and storing the result, is called the epilogue.

GPU Execution Hierarchy

A conventional CUDA execution hierarchy is

$$\text{grid} \rightarrow \text{thread blocks} \rightarrow \text{warps} \rightarrow \text{threads}.$$

Each thread block is scheduled onto an SM. Threads inside a block can cooperate through shared memory and synchronize with block-level barriers.

Hopper adds a new level:

$$\text{grid} \rightarrow \text{thread block clusters} \rightarrow \text{thread blocks} \rightarrow \text{warps} \rightarrow \text{threads}.$$

A thread block cluster is a group of thread blocks that can be co-scheduled onto a GPU processing cluster. This enables distributed shared memory, faster cluster-level synchronization, and multicast data movement.

GPU Memory Hierarchy

The relevant memory hierarchy for high-performance GEMM is

$$\text{global memory} \rightarrow \text{L2 cache} \rightarrow \text{shared memory} \rightarrow \text{registers} \rightarrow \text{Tensor Cores}.$$

Global memory has high capacity but high latency. Shared memory has lower latency and programmer-managed reuse. Registers are the fastest per-thread storage. Tensor Cores consume operands from registers or shared memory depending on the architecture.

Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

For GEMM, the total floating-point work is approximately

$$2MNK.$$

A tiled GEMM increases arithmetic intensity by loading tiles of A and B once and reusing them many times before writing the final output tile.

Main Concepts

What Tensor Cores Are

Tensor Cores are hardware units specialized for matrix multiply-accumulate operations. At a high level, they compute

$$D = AB + C.$$

The actual instruction shape depends on the GPU architecture and data type. Tensor Cores exploit the reuse structure of matrix multiplication. A tile of A is reused across many columns of B , and a tile of B is reused across many rows of A .

Evolution of Tensor Core Instructions

Before Tensor Cores

Before Tensor Cores, GEMM kernels used scalar or vector CUDA core instructions. Each thread performed multiply-add operations such as

$$c \leftarrow c + ab.$$

Performance depended heavily on shared-memory tiling and memory coalescing.

Pascal Dot Product Instructions

Pascal introduced lower-precision dot product instructions. These instructions allowed a thread to compute packed low-precision dot products, but they were not yet the cooperative Tensor Core model.

Volta Tensor Cores

Volta introduced the first NVIDIA Tensor Cores. A small group of threads cooperated to compute a matrix multiply-accumulate instruction. A representative shape was

$$8 \times 8 \times 4.$$

This increased throughput but introduced complicated thread-to-data ownership patterns.

Ampere Tensor Cores

Ampere used warp-level MMA instructions. A full warp of 32 threads cooperates on an instruction. A representative instruction shape is

$$16 \times 8 \times 8.$$

This corresponds to

$$A_{\text{frag}} \in \mathbb{R}^{16 \times 8}, \quad B_{\text{frag}} \in \mathbb{R}^{8 \times 8}, \quad C_{\text{frag}} \in \mathbb{R}^{16 \times 8}.$$

Operands are typically held in registers before being consumed by the Tensor Core instruction.

Hopper Tensor Cores

Hopper introduces warp-group MMA, often called WGMMA. A warp group contains four warps:

$$4 \times 32 = 128 \text{ threads.}$$

A representative Hopper instruction shape is

$$64 \times 16 \times 16.$$

Thus,

$$A_{\text{frag}} \in \mathbb{R}^{64 \times 16}, \quad B_{\text{frag}} \in \mathbb{R}^{16 \times 16}, \quad C_{\text{frag}} \in \mathbb{R}^{64 \times 16}.$$

The major Hopper change is that WGMMA can read operands directly from shared memory using descriptors. This removes some of the register-loading overhead present in earlier architectures.

Why Tensor Core Programming Is Difficult

The Partitioning Problem

A logical GEMM coordinate is simple:

$$(m, n, k).$$

A real GPU kernel must map it to

$$(\text{CTA}, \text{warp}, \text{warp group}, \text{thread}, \text{value}, \text{memory address}).$$

The mapping is architecture-specific. Tensor Core instructions require operands to be arranged in specific thread and memory layouts. A simple row-major mental model is not enough.

The challenge is

logical tensor coordinate $\rightarrow$ thread ownership $\rightarrow$ shared-memory layout $\rightarrow$ Tensor Core operand layout.

The Asynchrony Problem

Tensor Cores are so fast that a kernel cannot load data from global memory only when it is needed. The latency would dominate. Instead, the kernel must use a software pipeline:

$$\text{load tile } k + 1 \quad \text{while computing tile } k.$$

With S pipeline stages, shared memory contains

$$\text{stage}_0, \text{stage}_1, \dots, \text{stage}_{S-1}.$$

The active stage for iteration t is

$$s(t) = t \bmod S.$$

The kernel must ensure that producers do not overwrite a stage before consumers finish reading it, and consumers do not read a stage before producers finish writing it.

CUTLASS and CuTe

CUTLASS

CUTLASS is NVIDIA's C++ template library for high-performance linear algebra on GPUs. It provides reusable components for GEMM, convolutions, tensor contractions, epilogues, and custom fused kernels.

Its purpose is to expose a tile-based GPU programming model while hiding much of the architecture-specific complexity.

CuTe

CuTe is a core layout algebra library inside CUTLASS 3. A CuTe layout is represented as

$$\text{layout} = (\text{shape}, \text{stride}).$$

For a row-major matrix with shape (M, N) , the stride is

$$(N, 1),$$

so

$$\text{index}(m, n) = mN + n.$$

For a column-major matrix, the stride is

$$(1, M),$$

so

$$\text{index}(m, n) = m + nM.$$

CuTe generalizes this idea to hierarchical layouts, swizzled layouts, Tensor Core layouts, and thread layouts.

Why CuTe Matters

Earlier CUTLASS versions used many handwritten layout iterators. CuTe replaces these with a general algebra of layouts. This makes it possible to describe thread layouts and data layouts separately, then combine them through partitioning.

The key idea is

tensor $\rightarrow$ tile $\rightarrow$ partition $\rightarrow$ thread-owned fragment.

Swizzling

A swizzle is a transformation of memory layout used to avoid shared-memory bank conflicts. Conceptually,

$$\text{physical offset} = \text{swizzle}(\text{logical coordinate}).$$

On Hopper, TMA can write data into shared memory in a swizzled format, and WGMMMA can read that format efficiently.

Visual Concept

Draw a logical tile on the left and shared-memory banks on the right. First show a naive row-major mapping where many accesses collide on the same bank. Then show a swizzled mapping where accesses are distributed across banks.

CUTLASS 3 Hierarchy

Atom Layer

The atom layer represents the smallest hardware-supported operation, such as an MMA instruction or copy instruction.

An MMA atom defines:

- the instruction shape,
- participating threads,
- operand layouts,
- accumulator layout.

Tiled MMA and Tiled Copy Layer

This is the spatial microkernel layer. It tiles an atom across a larger tile.

If one atom computes

$$M_a \times N_a \times K_a,$$

then a tiled MMA composes atoms to cover

$$B_M \times B_N \times B_K.$$

Collective Layer

This is the temporal microkernel layer. It orchestrates tiled copies and tiled MMA operations across the K -dimension.

A collective handles:

- software pipelining,
- asynchronous copies,
- WGMMA instructions,
- producer-consumer barriers,
- shared-memory stage reuse.

Kernel Layer

The kernel layer maps the global problem onto CTAs or clusters. It composes:

$$\text{kernel} = \text{tile scheduler} + \text{mainloop collective} + \text{epilogue collective}.$$

It handles grid planning, persistent scheduling, stream- K , split- K , and tile swizzling for locality.

Hardware-Level Explanation

Thread Block Clusters

Hopper thread block clusters allow CTAs to be co-scheduled on nearby SMs. This enables cluster-level cooperation.

If a cluster has C CTAs, then the cluster can share data through distributed shared memory:

$$\text{DSM} = \bigcup_{i=0}^{C-1} \text{shared memory}_i.$$

Tensor Memory Accelerator

The Tensor Memory Accelerator, or TMA, performs asynchronous tensor-aware copies between global memory and shared memory.

A conceptual TMA copy is

$$A_{\text{smem}}[:, :, s] \leftarrow A_{\text{global}}[m_0 : m_0 + B_M, k_0 : k_0 + B_K].$$

TMA understands tensor shape, stride, element type, bounds, swizzle, and multicast information. This reduces manual address arithmetic and predicate computation.

TMA Multicast

If several CTAs in a cluster need the same tile, TMA multicast can load the tile once and distribute it to multiple shared-memory regions.

If r CTAs need the same tile, ideal traffic decreases from

$$r \cdot \text{Bytes}_{\text{tile}}$$

to approximately

$$\text{Bytes}_{\text{tile}}.$$

This improves effective L2 bandwidth.

Asynchronous Barriers

Hopper barriers can track both arrival counts and transaction counts. A barrier becomes ready when the required threads have arrived and the required memory transactions have completed.

$$\text{ready} \iff \text{arrival count complete} \quad \text{and} \quad \text{transaction count complete}.$$

This is essential because TMA and WGMMMA are asynchronous.

Producer-Consumer Pipeline

A Hopper GEMM often uses producer warps and consumer warp groups.

$$\text{producer warps} : \text{global memory} \rightarrow \text{shared memory}.$$

$$\text{consumer warp groups} : \text{shared memory} \rightarrow \text{Tensor Cores} \rightarrow \text{register accumulators}.$$

The pipeline is

$$\text{acquire} \rightarrow \text{TMA copy} \rightarrow \text{barrier signal} \rightarrow \text{WGMMMA wait} \rightarrow \text{WGMMMA compute} \rightarrow \text{release}.$$

Mathematical Reasoning

Tiled GEMM

Let the tile sizes be B_M , B_N , and B_K . One CTA computes an output tile

$$C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N}.$$

The tile is accumulated as

$$C_{\text{tile}} = \sum_{p=0}^{K/B_K-1} A_p B_p,$$

where

$$A_p \in \mathbb{R}^{B_M \times B_K}, \quad B_p \in \mathbb{R}^{B_K \times B_N}.$$

FLOPs Per Tile Step

For one K -tile step,

$$\text{FLOPs}_{\text{step}} = 2B_M B_N B_K.$$

Bytes Per Tile Step

If each element uses s bytes, the bytes loaded are

$$\text{Bytes}_{\text{step}} = s(B_M B_K + B_K B_N).$$

Thus arithmetic intensity for one tile step is

$$\text{AI}_{\text{step}} = \frac{2B_M B_N B_K}{s(B_M B_K + B_K B_N)}.$$

Factoring out B_K ,

$$\text{AI}_{\text{step}} = \frac{2B_M B_N}{s(B_M + B_N)}.$$

This shows that larger M - and N -tiles increase reuse, but they also increase register and shared-memory pressure.

Pipeline Stage Memory Cost

With S pipeline stages, shared-memory usage is approximately

$$\text{SMEM} = S \cdot s(B_M B_K + B_K B_N).$$

Increasing S improves latency hiding but consumes more shared memory.

$$S \uparrow \Rightarrow \text{better latency hiding,}$$

but

$$S \uparrow \Rightarrow \text{lower possible occupancy.}$$

Latency Hiding Condition

If global-memory latency is L_{mem} and one compute step takes L_{mma} , then a rough condition for hiding memory latency is

$$SL_{\text{mma}} \gtrsim L_{\text{mem}}.$$

If

$$SL_{\text{mma}} < L_{\text{mem}},$$

the consumer catches the producer and stalls.

Code Walkthrough

Naive GEMM Baseline

```
for (int m = 0; m < M; ++m) {
    for (int n = 0; n < N; ++n) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k) {
            acc += A[m * K + k] * B[k * N + n];
        }
        C[m * N + n] = acc;
    }
}
```

This code is mathematically clear, but it does not expose GPU parallelism, shared-memory reuse, or Tensor Core instructions.

Tiled GEMM Structure

```
int block_m = blockIdx.y;
int block_n = blockIdx.x;

int m0 = block_m * BlockM;
int n0 = block_n * BlockN;

for (int k0 = 0; k0 < K; k0 += BlockK) {
    // Load A and B tiles into shared memory.
    // Synchronize.
    // Compute partial products.
    // Synchronize before reusing shared memory.
}
```

A high-performance Tensor Core kernel replaces the comments with TMA copies, swizzled shared-memory layouts, WGMMMA instructions, and asynchronous barriers.

CUTLASS Collective Builder

```
using CollectiveMainloop =
    cutlass::gemm::collective::CollectiveBuilder<
        ArchTag,
        OperatorClass,
        ElementA,
        LayoutA,
        AlignmentA,
        ElementB,
        LayoutB,
        AlignmentB,
        ElementAccumulator,
        TileShape,
        ClusterShape,
        StageCountAuto,
        KernelScheduleAuto
    >::CollectiveOp;
```

The builder receives architecture-independent parameters and chooses architecture-specific details such as the MMA atom, shared-memory layout, TMA strategy, and pipeline depth.

Epilogue Fusion

A fused epilogue may compute

$$D_{m,n} = \max(0, \alpha \cdot \text{Acc}_{m,n} + \text{bias}_n + \beta C_{m,n}).$$

A Python-like expression is

```
def epilogue(acc, C, bias, alpha, beta):  
    x = alpha * acc  
    x = x + bias  
    x = x + beta * C  
    return relu(x)
```

CUTLASS can represent this as an epilogue visitor tree, allowing fusion without rewriting the main Tensor Core kernel.

Hopper Pipeline Pseudocode

```
for (int tile_k = 0; tile_k < num_k_tiles; ++tile_k) {  
    int stage = tile_k % Stages;  
  
    producer_acquire(stage);  
  
    issue_tma_copy_A(stage, tile_k);  
    issue_tma_copy_B(stage, tile_k);  
  
    producer_commit(stage);  
  
    consumer_wait(stage);  
  
    warpgroup_arrive();  
  
    wgmma(accumulator, shared_A(stage), shared_B(stage));  
  
    warpgroup_wait();  
  
    consumer_release(stage);  
}
```

The important behavior is that TMA copies and WGMMMA computation are asynchronous. Correct barriers are required to prevent races.

Performance Intuition

Why TMA Matters

TMA reduces instruction overhead by moving multidimensional tensor-copy logic into hardware. It avoids excessive per-thread address arithmetic such as

$$\text{address} = \text{base} + i \cdot \text{stride}_i + j \cdot \text{stride}_j.$$

This leaves more instruction bandwidth and scheduling capacity for useful work.

Why Software Pipelining Matters

Without pipelining,

load $\rightarrow$ wait $\rightarrow$ compute $\rightarrow$ load $\rightarrow$ wait $\rightarrow$ compute.

With pipelining,

load tile $k + 1$ $\parallel$ compute tile k .

The goal is to keep Tensor Cores busy.

Why Warp Specialization Matters

Warp specialization assigns different roles to different warps. Some warps issue TMA loads, while others issue WGMMMA instructions. This increases overlap and reduces exposed latency, but it requires careful synchronization.

Why Persistent Scheduling Matters

Persistent kernels let a CTA compute multiple output tiles. If setup cost is T_{setup} and a CTA computes q tiles, the setup cost per tile is

$$\frac{T_{\text{setup}}}{q}.$$

This can improve performance when fixed overheads are significant.

Common Misconceptions

Tensor Cores Automatically Guarantee High Performance

Tensor Cores only provide high peak throughput. Actual performance depends on memory movement, synchronization, tiling, and layout.

GEMM Is Just Three Loops

The formula is simple, but the GPU implementation is a hierarchy of tiled loops, thread ownership mappings, shared-memory layouts, and asynchronous pipelines.

Occupancy Is Always the Main Metric

Occupancy matters, but Tensor Core kernels often care more about whether Tensor Cores are continuously fed. A lower-occupancy kernel with better pipelining can outperform a high-occupancy kernel with stalls.

Shared Memory Is Automatically Fast

Shared memory can be slow if accesses cause bank conflicts or if synchronization is poorly managed.

CUTLASS Is Only for Library GEMMs

CUTLASS is also a framework for writing custom kernels by reusing optimized collectives, epilogues, and schedulers.

Practical Takeaways

- Think of Tensor Core programming as dataflow orchestration, not just matrix multiplication.
- Use CuTe layouts to avoid manual index arithmetic.
- Use TMA on Hopper for efficient global-to-shared and shared-to-global tensor movement.
- Use WGMMA to target Hopper Tensor Cores.
- Use asynchronous barriers to coordinate producers and consumers.
- Tune tile shape, pipeline depth, cluster shape, and schedule.
- Use CUTLASS profiler to search the kernel design space.
- Use epilogue visitor trees for fused post-processing.

Project or Experiment Ideas

Naive GEMM Versus Tiled GEMM

Implement a naive GEMM and a tiled shared-memory GEMM. Compare runtime, memory throughput, and achieved FLOPs.

Pipeline Depth Study

Benchmark different pipeline depths:

$$S \in \{2, 3, 4, 5, 6\}.$$

Measure when deeper staging helps and when it reduces occupancy too much.

Epilogue Fusion

Compare a separate GEMM plus bias plus ReLU pipeline against a fused CUTLASS epilogue.

Triton Versus CUTLASS

Implement the same fused GEMM in Triton and CUTLASS. Compare development effort and achieved performance.

Shared-Memory Layout Experiment

Compare a naive shared-memory layout against a swizzled or padded layout and inspect shared-memory bank conflicts in Nsight Compute.

Final Mental Model

A high-performance Tensor Core GEMM is a pipeline. Global memory provides tiles. TMA moves those tiles into swizzled shared memory. Barriers track when the tiles are ready. WGMMMA consumes the tiles using warp groups and accumulates results in registers. The epilogue transforms the accumulators into final outputs. CUTLASS and CuTe exist to make this pipeline programmable, composable, and correct.

The core mental model is that Tensor Cores are hungry matrix engines. CUTLASS builds the feeding system, CuTe describes where data lives, TMA moves tiles into place, barriers coordinate the pipeline, WGMMMA performs the matrix math, and the epilogue turns accumulators into useful output.